

PROJECT PORTFOLIO

젤다의 전설(야생의 숨결)

6인 DirectX 프로젝트에서 톤매핑·SSAO·캐스케이드 그림자와 라이넬·가논 보스 전투를 담당했다.



지형 장면의 SSAO 적용 비교

프로젝트	6인 협업
담당 역할	톤매핑·SSAO·캐스케이드 그림자, 라이넬·가논 보스 전투
기술	C++, DirectX, HLSL, ImGui, JSON

<https://yollo4179.github.io/projects/zelda-breath-of-the-wild/>

내가 구현한 영역

톤매핑·SSAO·캐스케이드 그림자, 라이넬·가논 보스 전투

6명이 함께 만든 DirectX 기반 게임 프로젝트다. 내가 맡은 범위는 조명·그림자 표현과 보스 몬스터 두 마리다. 화면 밝기는 평균 휘도를 이용한 톤매핑으로 계산했다. 물체가 맞닿는 영역의 음영은 SSAO로 계산했고, 넓은 시야의 그림자는 카메라 거리별로 나눠 처리했다.

전투 영역에서는 라이넬과 가논의 상태 흐름, 공격 오브젝트와 풀링을 구현했다.

담당 구현

- **렌더링:** 컴퓨터 셰이더의 평균 휘도 축소, Y_{xy} 기반 톤매핑, SSAO 샘플·깊이 비교, 캐스케이드 그림자.
- **보스 전투:** 라이넬·가논의 상태 전환과 공격 구성, 공격 오브젝트의 내부 도메인 풀링, 가논의 벽·천장 행렬 회전, 라이넬의 소켓 전환·기절·2페이지 후속 공격.

기술 구성

C++, DirectX, HLSL, ImGui, JSON

실행 화면과 구현 코드에 연결된 블로그·영상 링크는 PDF에서 클릭해 열 수 있습니다.

기능별 구현

01	평균 휘도 축소와 톤매핑	04
02	깊이·노멀 버퍼로 SSAO 계산하기	07
03	카메라 거리별 그림자 영역 분할	13
04	라이넬·가논의 내부 도메인 풀링과 공격 수명 관리	18
05	가논 — 거리 기반 공격과 벽·천장 전환의 행렬 회전	21
06	라이넬 — 패턴 순환과 소켓 전환, PhysX 이동과 기절 처리	28

01 / 화면의 밝기와 음영

평균 휘도 축소와 톤매핑

로그 휘도의 2×2 축소, 프레임 간 휘도 적응과 Yxy 톤 커브로 화면 밝기를 계산한다.

[블로그 원문 ↗](#)

톤매핑 적용 영상 1 · [YouTube에서 보기](#)

톤매핑 적용 영상 2 · [YouTube에서 보기](#)

구현 구조

TEXT

```
Client/Private/HDMI_Manager.cpp
Client/Bin/ShaderFiles/CShader_Deffered_HDRI.hlsl
Client/Bin/ShaderFiles/Shader_Deffered_HDRI.hlsl
```

HDR 색 → 로그 휘도 → 2×2 평균 축소 → 이전 휘도와 시간 보간 → Yxy 톤 커브 → RGB 출력.

로그 휘도를 2×2 평균으로 축소

`CS_Luminance()`는 RGB에 휘도 가중치를 적용하고 `log(max(luminance, 1e-6))`를 기록한다. `CS_DownScale()`의 각 스레드는 입력 네 픽셀의 평균을 출력한다. 매니저는 축소 단계마다 이전 SRV와 다음 UAV를 바인딩해 평균 휘도를 줄여 나간다.

`Client/Bin/ShaderFiles/CShader_Deffered_HDRI.hlsl` 발췌

HLSL

```
void CS_DownScale(uint3 DTid : SV_DispatchThreadID)
{
    float2 texCoord = (float2(DTid.xy))*2;
    float Avg = 0;
    for (int i = 0; i < 2;++i)
        for (int j = 0; j < 2;++j)
        {
            Avg += InputTexture.Load(int3(texCoord + uint2(i,j), 0));
        }
    Avg *= 0.25f;
    OutputReturn[DTid.xy] = Avg;
}
```

이전 프레임과 밝기 적응

축소 결과를 `exp()`로 복원한다. 적응 속도와 프레임 시간으로 계산한 alpha를 사용해 이전 휘도와 새 휘도를 섞는다. 필터 결과는 다음 프레임의 이전 휘도로 복사한다.

평균 휘도 축소와 톤매핑 · 계속

Client/Bin/ShaderFiles/Shader_Deffered_HDRI.hlsl 발췌

```

HLSL

PS_OUT PS_MAIN_FILTER_LUM(PS_IN In)
{
    PS_OUT Out = (PS_OUT) 0;
    float alpha = 1.0f - exp(g_AdaptationSpeed * (-g_Time));
    float fDstLum = g_AvgLumNewTexture.Sample(LinearSampler, In.vTexcoord).x;
    fDstLum = exp(fDstLum);
    float fSrcLum = (g_AvgLumPrevTexture.Sample(LinearSampler, In.vTexcoord).x);
    float fUpdatedLum = (1.f - alpha) * fSrcLum + (alpha) * fDstLum;
    Out.vColor.x = fUpdatedLum;
    return Out;
}

```

Yxy 휘도에 톤 커브 적용

RGB를 Yxy로 변환하고 밝기 Y에 평균 휘도와 MiddleGrey를 적용한다. Reinhard 커브에 WhitePoint 매개변수를 전달한 뒤 RGB로 되돌린다. 색좌표와 밝기를 분리해 노출과 톤 커브를 처리한다.

Client/Bin/ShaderFiles/Shader_Deffered_HDRI.hlsl 발췌

```

HLSL

float3 NewHDR2(float g_AvgLuminance, float3 vHDRColor)
{
    float AvgLuminance =g_AvgLuminance;
    float3 Yxy = ConvertRGB2Yxy(vHDRColor);
    float lp = Yxy.x * (g_MiddleGrey / (AvgLuminance*9.6 + 0.0001));
    Yxy.x = reinhard2(lp, g_fLumWhiteSqr);
    float3 mappedRGB = convertYxy2RGB(Yxy);
    return mappedRGB.xyz;
}

```

톤매핑 영상과 수식 자료

```

Texture2D<float> Pass3_InputData : register(t0);
RWTexture2D<float> Pass3_TileSum : register(u0);

#define LUMINANCE_FACTOR float3(0.2126, 0.7152, 0.0722)
#define ROWID 16
#define GROUPSIZE 256

//Pass1_InputData -> Make Tiles(16*16) // //2D_Reduction
groupshared float g_LogLum[256]; //XYZ
[numthreads(16, 16, 1)]
void CS_ParelellReductionForAvgLum(
    uint3 DTid : SV_DispatchThreadID,
    uint3 GTid : SV_GroupThreadID,
    uint Gidx : SV_GroupIndex,
    uint3 Gid : SV_GroupID)
{
    float3 vRGB = Pass1_InputData.Load(int3(DTid.xy, 0));
    float fYyLum = dot(vRGB, LUMINANCE_FACTOR);
    unsigned int iMyGroupIndex = GTid.x + ROWID * GTid.y; //=- Gidx
    g_LogLum[iMyGroupIndex] = log(max(fYyLum, 1e-6));
    //Making Sure We fill All Data From Input Texture To Group
    GroupMemoryBarrierWithGroupSync();

    for (int iOffset = GROUPSIZE / 2; iOffset > 0; iOffset >>= 1) //Half
    {
        if (iMyGroupIndex < iOffset) //~127 ~64 ~32... / 2->1(Completed to Get Group Avg)
            g_LogLum[iMyGroupIndex] += g_LogLum[iMyGroupIndex + iOffset];
        // Accumulate Half of the datum other side linearly Every Execution
        GroupMemoryBarrierWithGroupSync();
        //Move on to the next tree level
    }

    //After All, The result is in The First position of shared group memory with Prefix Sum
    if (0 == GTid.x && 0 == GTid.y)
    {
        Pass3_TileSum[Gid.xy] = g_LogLum[0] / 256; // [Gid]- (Width/16) * (Height/16) ==size ) GTid(0-16,0-16) -> DTid ~Gid * Size +DTid
    }
}

//1024 <= (1920/16)*(1080/16)= 120 *68 =6800+ 680*2= 8160 */
// OutputTexture -> 120 *68크기

```

컴퓨터 셰이더의 휘도 병렬 리덕션 코드

평균 휘도 축소와 톤매핑 · 계속

```
float reinhard2(float L, float whitePoint)
{
    float numerator = L * (1.0 + (L / (whitePoint * whitePoint)));
    float denominator = 1.0 + L;
    return numerator / denominator;
}
```

평균 휘도를 이용한 밝기 보정 코드

```
float3 ToneMapping2(float AvgLuminance, float3 vHDRColor) //평균휘도, 픽셀 색상
{
    // 현재 픽셀에 대응되는 톤매핑 조정휘도로 최종 색상을 결정한다.
    float3 Yxy = ConvertRGB2Yxy(vHDRColor); //Yxy 색상공간 float3(Y,x,y) Y 값은 현재 픽셀 색상의 휘도값이다.
    float Ls = Yxy.x * (g_MiddleGrey / (AvgLuminance*9.6 + 0.0001)); //MiddleGrey가 높으면 휘도가 높아진다.
    Yxy.x = reinhard2(Ls, g_fLumWhiteSqr); //톤커브 공식 적용해 새로운 휘도를 구한다.
    return convertYxy2RGB(Yxy); //새로운 휘도값을 바탕으로 색상을 재설정한다.
}
```

Yxy 색공간 변환 코드

```
PS_OUT PS_MAIN_END_TONEMAPPING(PS_IN In)
{
    PS_OUT Out = (PS_OUT) 0;
    float3 vBloomColor = (Gamma_LinearSpace(g_BloomTexture.Sample(Linearsampler, In.vTexcoord))); //빌연산을 완료한 렌더 타겟으로부터 임계휘도를 넘은 색상만 적어놓은 블룸 마스크 렌더타겟
    float fLum = g_AvgLuminanceTexture.Sample(Linearsampler, In.vTexcoord).x; //구해놓은 1 x 1 사이즈의 평균휘도 샘플링한다.
    Out.vColor.xyz = Gamma_ForMonitor(ToneMapping2(fLum, (vBloomColor + g_EndTargetTexture.Sample(Linearsampler, In.vTexcoord).xyz))); // 톤매핑을 적용한 최종 픽셀의 색상을 결정
    return Out;
}
```

최종 색상을 계산하는 코드

$$L_{scaled} = \frac{L \cdot middleGrey}{L_{avg}}$$

MiddleGrey와 평균 휘도의 관계식

$$L_{out} = \frac{L_{scaled} \cdot \left(1 + \frac{L_{scaled}}{WhitePoint^2}\right)}{1 + L_{scaled}}$$

WhitePoint를 반영한 톤 커브 식

02 / 화면의 밝기와 음영

깊이·노멀 버퍼로 SSAO 계산하기

깊이·노멀에서 뷰 공간 정보를 복원하고 깊이 비교와 양방향 블러를 계산하는 SSAO 셰이더를 구성했다.

[블로그 원문 ↗](#)

SSAO 적용 영상 · [YouTube에서 보기](#)

구현 구조

TEXT

Client/Bin/ShaderFiles/CShader_Deffered_SSAO.hlsl

깊이·노멀 텍스처 읽기 → 뷰 공간 위치와 노멀 복원 → 깊이 비교 값 누적 → 차폐값 출력 → 깊이를 고려한 블러.

깊이와 노멀을 뷰 공간으로 복원

셰이더는 깊이·노멀 텍스처를 픽셀 좌표로 읽고, 노멀을 -1~1 범위로 복원해 뷰 공간으로 변환한다. 위치는 화면 UV와 깊이 값에서 역투영한다. 화면 크기는 1920×1080, 깊이 복원에 사용하는 Far 값은 1000으로 고정돼 있다.

[Client/Bin/ShaderFiles/CShader_Deffered_SSAO.hlsl](#) 발췌

HLSL

```
uint2 TexSize = { 1920, 1080 };
float2 TexCoord = float2(DTid.xy) / float2(TexSize);
vector vDepth = InputDepthTexture.Load(int3(DTid.xy, 0));
float2 NoiseScale = { 1920 / 4.F, 1080 / 4.F };
vector vNormal = InputNormalTexture.Load(int3(DTid.xy, 0));
vNormal.xyz = normalize(vNormal.xyz * 2.f - 1.f);
float3x3 matView3x3 = (float3x3) gView;
float3 vNormal_InViewSpace = normalize(mul(vNormal.xyz, matView3x3));
vector vPosition = (vector) 0;
vPosition.x = TexCoord.x * 2.f - 1.f;
vPosition.y = TexCoord.y * -2.f + 1.f;
vPosition.z = vDepth.x;
vPosition.w = 1.f;
vPosition *= vDepth.y * 1000;
vPosition = mul(vPosition, gInvProjection);
```

커널과 깊이 비교

셰이더는 노이즈 벡터와 뷰 공간 노멀로 TBN 행렬을 구성하고, 커널에 반경과 0.8을 곱해 샘플 위치를 계산한다. 샘플 위치를 투영한 UV에서 깊이를 읽고, 해당 깊이가 sample.z - gBias - 0.2f보다 작으면 차폐 횟수를 늘린다. 최종 값은 1 - occlusion / gNumSamples로 출력한다.

깊이·노멀 버퍼로 SSAO 계산하기 · 계속

Client/Bin/ShaderFiles/CShader_Deffered_SSAO.hlsl 발췌

HLSL

```
float3 RandomVec = normalize(SSAONoiseTexture.SampleLevel(gSampler, NoiseScale * TexCoord, 0)).xyz;
float3 tangent = normalize(RandomVec - vNormal_InViewSpace * dot(RandomVec, vNormal_InViewSpace));
float3 bitangent = cross(vNormal_InViewSpace, tangent);
float3x3 TBN = float3x3(tangent, bitangent, vNormal_InViewSpace);
float occlusion = 0.f;
for (int i = 0; i < gNumSamples; ++i)
{
    float3 sample = mul(gSampleKernel[i].xyz, TBN);
    sample = vPosition.xyz + sample * gRadius*0.8f;
    vector offset = vector(sample, 1.f);
    offset = mul(offset, gProjection);
    offset.x /= offset.w;
    offset.y /= offset.w;
    offset.x = offset.x * 0.5f + 0.5f;
    offset.y = offset.y * -0.5f + 0.5f;
    float OccluderPosZ = InputDepthTexture.SampleLevel(gSampler, offset.xy, 0).y * 1000.f;
    occlusion += (OccluderPosZ < sample.z - gBias-0.2f ? 1.f : 0.f);
}
float ao = 1.f - (occlusion / gNumSamples);
OutputReturnSSAO[DTid.xy] = ao;
```

TBN 변환: tangent-bitangent-뷰 공간 노멀을 각 행으로 구성한다. mul(gSampleKernel[i].xyz, TBN)은 커널의 X·Y·Z 성분을 각 기저에 곱해 더해 접선 공간 샘플을 뷰 공간 방향으로 변환한다. 반경을 곱하고 vPosition.xyz를 더하면 깊이 비교에 사용할 샘플 위치가 된다.

깊이 차이를 반영한 양방향 블러

블러는 가로·세로 방향으로 나눠 처리한다. 화면상 거리에 따른 가중치와 중심·이웃 픽셀의 깊이 차이에 따른 가중치를 곱하고, 가중치 합으로 나눠 출력한다. 깊이가 다른 표면은 블러에 기여하는 비중이 작아진다.

Client/Bin/ShaderFiles/CShader_Deffered_SSAO.hlsl 발췌

HLSL

```
float BilateralWeight(float centerDepth, float sampleDepth, float sigma)
{
    float diff = centerDepth - sampleDepth;
    return exp(-diff * diff / (2 * sigma * sigma));
}
```

깊이·노멀 버퍼로 SSAO 계산하기 · 계속

Client/Bin/ShaderFiles/CShader_Deffered_SSAO.hlsl 발췌

```
HLSL
if (0.5f < Vertical_Horize.x)
{
    for (int i = -fRadius; i <= fRadius; ++i)
    {
        float2 UVAndoffset = float2(vTexCoord.x + (1.f / texSize.x) * i, vTexCoord.y);
        float SampleDepth = DepthTexture.SampleLevel(gSampler, UVAndoffset, 0).x;
        float SampleSSAO = InputTexture.SampleLevel(gSampler, UVAndoffset, 0).x;
        float spatialWeight = exp(-i * i / (2 * sigmaD * sigmaD));
        float rangeWeight = BilateralWeight(fCenterDepth.x, SampleDepth, sigmaR);
        float weight = spatialWeight * rangeWeight;
        result += SampleSSAO.x * weight;
        weightSum += weight;
    }
}
```

SSAO 영상과 계산 자료



지형 장면의 SSAO 적용 비교

깊이·노멀 버퍼로 SSAO 계산하기 · 계속



풀밭 장면의 SSAO 적용 비교

```

HRESULT CSSAO_Manager::Ready_Kernels()
{
#define LERP( Src, Dst, Factor) Src*(1-Factor) +(Dst)*Factor;
    _float3 vKernels[64] = {};
    for (unsigned int i = 0; i < 64; ++i)
    {
        _float X = m_pGameInstance->Compute_Random(0.f, 1.f) * 2.f - 1.f; //T
        _float Y = m_pGameInstance->Compute_Random(0.f, 1.f) * 2.f - 1.f; //B
        _float Z = m_pGameInstance->Compute_Random(0.f, 1.f); //N (RandomNormal
        _vector vKernel = { X,Y,Z, 0.f };
        vKernel = XMVector3Normalize(vKernel);
        vKernel *= m_pGameInstance->Compute_Random(0.f, 1.f);
        //Scale Samples sothat thry cluster near origin
        float scale = float(i) / 64.f;
        vKernel *= LERP(0.1f, 1.f, scale * scale);
        XMStoreFloat3(&m_Params.g_SampleKernel[i], vKernel);
    }

    return S_OK;
}

```

노멀 주변 반구 샘플 생성 코드

깊이-노멀 버퍼로 SSAO 계산하기 · 계속

```
HRESULT CSSAO_Manager::Ready_SSAO_NoiseTexture()
{
    vector<_float3>ssaoNoiseTexture;
    D3D11_TEXTURE2D_DESC desc = {};
    desc.Width = 4;           // 가로 4 texel
    desc.Height = 4;          // 세로 4 texel
    desc.MipLevels = 1;
    desc.ArraySize = 1;
    desc.Format = DXGI_FORMAT_R32G32B32_FLOAT; // XMFLAOT3 * 3 float(12 B)
    desc.SampleDesc.Count = 1;
    desc.Usage = D3D11_USAGE_DYNAMIC; // CPU가 Map(WRITE_DISCARD) 가능
    desc.BindFlags = D3D11_BIND_SHADER_RESOURCE; // 픽셀-컴퓨터 셰이더에서 사용
    desc.CPUAccessFlags = D3D11_CPU_ACCESS_WRITE; // 쓰기 전용
    m_pDevice->CreateTexture2D(&desc, nullptr, &m_pSSAONoiseTexture2D);
    if (FAILED(m_pDevice->CreateShaderResourceView(m_pSSAONoiseTexture2D, nullptr, &m_pSSAONoiseSRV)))
        return E_FAIL;
    for (int i = 0; i < 16; ++i)
    {
        _float X = m_pGameInstance->Compute_Random(0.f, 1.f) * 2.f - 1.f; //T
        _float Y = m_pGameInstance->Compute_Random(0.f, 1.f) * 2.f - 1.f; //B
        _float Z = 0.f;
        ssaoNoiseTexture.push_back(_float3{ X, Y, 0 });
    }
    D3D11_MAPPED_SUBRESOURCE mappedResource = {};
    if (FAILED(m_pContext->Map(m_pSSAONoiseTexture2D, 0, D3D11_MAP_WRITE_DISCARD, 0, &mappedResource)))
        return E_FAIL;
    const uint8_t* src = reinterpret_cast<const uint8_t*>(ssaoNoiseTexture.data());
    uint8_t* dst = reinterpret_cast<uint8_t*>(mappedResource.pData);
    for (UINT y = 0; y < 4; ++y)
    {
        memcpy(dst + y * mappedResource.RowPitch,
            src + y * 4 * sizeof(DirectX::XMFLAOT3),
            4 * sizeof(DirectX::XMFLAOT3)); // 4픽셀 복사
    }
    //memcpy(mappedResource.pData, &ssaoNoiseTexture, 16*sizeof(DirectX::XMFLAOT3));
    m_pContext->Unmap(m_pSSAONoiseTexture2D, 0);
    return S_OK;
}
```

샘플 분산용 노이즈 텍스처 생성 코드

```
[numthreads(32, 32, 1)]
void CS_SSAO(uint3 DTid : SV_DispatchThreadID)
{
    /*깊이값과 Texcoord*/

    //InputDepthTexture.GetDimensions(TexSize.x, TexSize.y);
    uint2 TexSize = { 1920, 1080 };
    float2 TexCoord = float2(DTid.xy) / float2(TexSize);
    vector vDepth = InputDepthTexture.Load(int3(DTid.xy, 0)); // .X 에는 wvp.Z/wvp.W || .Y에는 wvp.w/Far --Far은 1000
    float2 NoiseScale = { 1920 / 4.f, 1080 / 4.f };

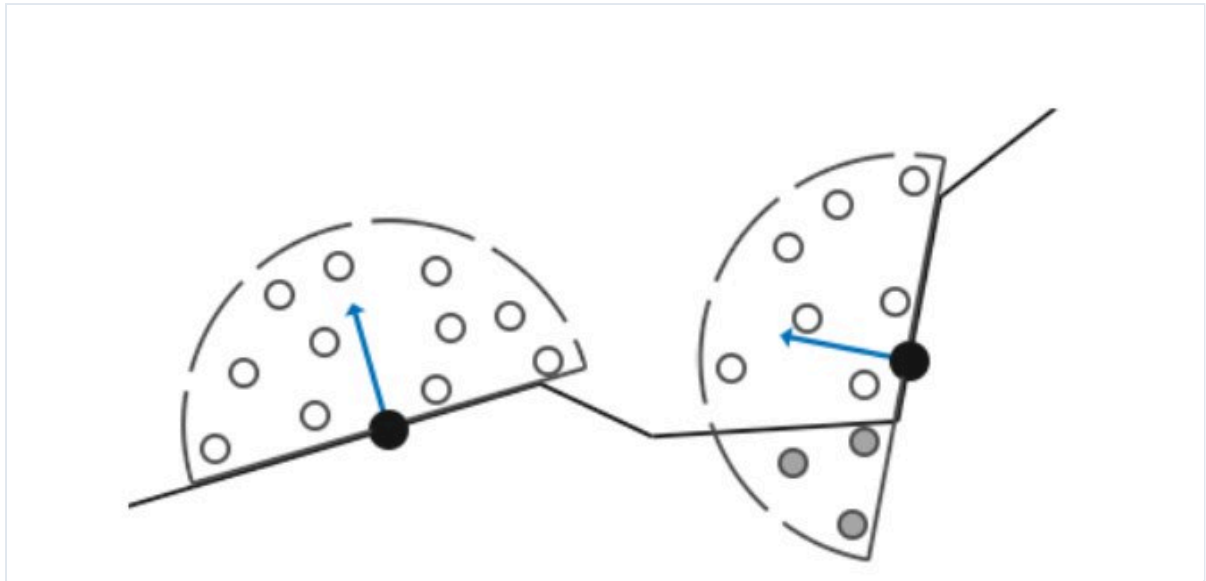
    /*뷰스페이스 상의 노말*/
    vector vNormal = InputNormalTexture.Load(int3(DTid.xy, 0));
    vNormal.xyz = normalize(vNormal.xyz * 2.f - 1.f);
    float3x3 matView3x3 = (float3x3) gView;
    float3 vNormal_InViewSpace = normalize(mul(vNormal.xyz, matView3x3));

    /*뷰스페이스 상의 공간*/
    vector vPosition = (vector) 0;
    /*Ndc*/
    vPosition.x = TexCoord.x * 2.f - 1.f;
    vPosition.y = TexCoord.y * 2.f + 1.f;
    vPosition.z = vDepth.x;
    vPosition.w = 1.f;

    /*ClipSpace */
    vPosition *= vDepth.y * 1000; // 1000은 Far
    /*ViewSpace*/
    vPosition = mul(vPosition, gInvProjection);
}
```

픽셀의 뷰 공간 위치 복원 코드

깊이·노멀 버퍼로 SSAO 계산하기 · 계속



SSAO 반구 샘플의 좌표 변환 설명

03 / 그림자

카메라 거리별 그림자 영역 분할

분할 프러스텀을 포함하는 그림자 카메라를 만들고 거리별 그림자 맵 깊이를 비교한다.

[블로그 원문 ↗](#)

캐스케이드 그림자 적용 영상 · [YouTube에서 보기](#)

구현 구조

TEXT

Engine/Private/CSM_Manager.cpp
Engine/Private/Renderer.cpp
Engine/Bin/ShaderFiles/Shader_Deferred.hlsl

카메라 프러스텀 분할 → 분할 영역의 월드 경계 계산 → 광원 뷰·직교 투영 생성 → 구간별 그림자 맵 → 깊이 비교.

카메라 시야를 세 구간으로 나누기

NDC의 여덟 모서리에 역투영·역뷰 행렬을 적용해 월드 프러스텀을 구한다. 각 구간의 near-far 범위로 모서리를 보간하고 경계 상자의 중심과 크기를 계산한다. 렌더러는 Near-Middle-Far 그림자 타킷에 각 구간의 행렬을 전달한다.

광원 방향으로 뷰 행렬 만들기

광원 카메라는 분할 영역의 중심을 바라본다. 광원 방향과 경계 상자의 Z 크기로 눈 위치를 정하고 `XMMatrixLookAtLH()`로 뷰 행렬을 만든다.

Engine/Private/CSM_Manager.cpp 발췌

CPP

```
HRESULT CSM_Manager::Compute_ViewMatrix()
{
    for (size_t i = 0; i < CSM_END; ++i)
    {
        _vector Center = XMVectorSet(
            m_BoundingAABBCorners[i].Center.x,
            m_BoundingAABBCorners[i].Center.y,
            m_BoundingAABBCorners[i].Center.z,
            1.f
        );
        _vector vEyePos = Center - 2 * XMVector3Normalize(XMVectorSetW(XMLoadFloat3(&m_vLightDir), 0)) * m_BoundingAABBCorners[i].Extents.z;
        _vector vUp = XMVectorSet(0.f, 1.f, 0.f, 0.f);
        XMStoreFloat4x4(&m_TransformMatrices[D3DTS_VIEW][i], XMMatrixLookAtLH(vEyePos, Center, vUp));
    }
    return S_OK;
}
```

카메라 거리별 그림자 영역 분할 · 계속

영역 크기로 직교 투영 구성

경계 상자의 X·Y 크기로 좌우·상하 범위를 설정하고, Z 크기로 깊이 범위를 만든다. 각 구간에 대응하는 직교 투영 행렬과 그림자 맵을 사용해 픽셀 깊이를 비교한다.

Engine/Private/CSM_Manager.cpp 발췌

C++

```
HRESULT CCSM_Manager::Compute_ProjMatrix()
{
    for (size_t i = 0; i < CSM_END; ++i)
    {
        _float left = - m_BoundingAABBCorners[i].Extents.x;
        _float right = + m_BoundingAABBCorners[i].Extents.x;
        _float bottom = - m_BoundingAABBCorners[i].Extents.y;
        _float top = m_BoundingAABBCorners[i].Extents.y;
        _float nearZ = - m_BoundingAABBCorners[i].Extents.z;
        _float farZ = + m_BoundingAABBCorners[i].Extents.z;
        XMStoreFloat4x4(&m_TransformMatrices[D3DTS_PROJ][i], XMMatrixOrthographicOffCenterLH(left, right, bottom, top, 0.f, farZ - nearZ));
    }
    return S_OK;
}
```

그림자 영상과 행렬 자료

```
HRESULT CCSM_Manager::Initialize()
{
    /* 절두체 구성을 위한 점8개 */
    /* NDC 공간-> 월드의 프러스텀*/
    m_vPoints[0] = _float3(-1.f, 1.f, 0.f);
    m_vPoints[1] = _float3(1.f, 1.f, 0.f);
    m_vPoints[2] = _float3(1.f, -1.f, 0.f);
    m_vPoints[3] = _float3(-1.f, -1.f, 0.f);

    m_vPoints[4] = _float3(-1.f, 1.f, 1.f);
    m_vPoints[5] = _float3(1.f, 1.f, 1.f);
    m_vPoints[6] = _float3(1.f, -1.f, 1.f);
    m_vPoints[7] = _float3(-1.f, -1.f, 1.f);

    return S_OK;
}
```

NDC 프러스텀 경계 정점 구성

카메라 거리별 그림자 영역 분할 · 계속

```

HRESULT CCSM_Manager::Compute_AABBFromCorners()
{
    for (size_t i = 0; i < CSM_END; ++i)
    {
        _float3 minCorner(FLT_MAX, FLT_MAX, FLT_MAX);
        _float3 maxCorner(-FLT_MAX, -FLT_MAX, -FLT_MAX);
        _vector vCenter = {};
        _vector vRadius = {};
        for (size_t j = 0; j < 8; ++j)
        {
            vCenter+=XMLoadFloat3(&m_vPoints_InWorld[i][j]);
        }
        _float3 CenterPos;
        XMStoreFloat3(&CenterPos, vCenter /= 8.f);

        _float fRadius = { 0 };
        for (size_t j = 0; j < 8; ++j)
        {
            _float fDistance = XMVectorGetX(XMVector3Length(XMLoadFloat3(&m_vPoints_InWorld[i][j]) - vCenter));
            fRadius = max(fRadius, fDistance);
        }
        fRadius = ceil(fRadius * 16.f) / 16.f;
        //Using RADIUS WeMade AABBBox

        _float3 Radius = _float3(fRadius, fRadius, fRadius);
        m_BoundingAABBcorners[i] = BoundingBox(CenterPos, Radius);
    }

    // we got aabb box from World Corners -> now We need to make Camera For
    // Shadow map
    return S_OK;
}

```

분할 프리스탐의 경계 상자 계산

```

HRESULT CCSM_Manager::Compute_FrustumCorners_World()
{
    //From Ndc-> Proj
    _matrix ViewMatrixInverse = m_pGameInstance->Get_TransformMatrix_Inverse(CPipeline::D3DTS_VIEW);
    _matrix ProjMatrixInverse = m_pGameInstance->Get_TransformMatrix_Inverse(CPipeline::D3DTS_PROJ);
    /*Now We're gonna get 8 FrustumPoints in World Space*/
    _vector vCorner[8] = {};
    for (size_t i = 0; i < 8; ++i)
    {
        /*Ndc->It Could Be Out from LOOP*/
        vCorner[i] = XMVectorSetW(XMLoadFloat3(&m_vPoints[i]), 1.f);
        vCorner[i] = XMVector3TransformCoord(vCorner[i], ProjMatrixInverse);
        /*Now We Are In View Space */
        vCorner[i] = XMVector3TransformCoord(vCorner[i], ViewMatrixInverse);
    }
    for (size_t i = 0; i < 4; ++i) {
        for (size_t j = 0; j < CSM_END; ++j) { CSM_NEAR, CSM_MIDDLE, CSM_FAR, CSM_END }
        {
            _vector vNearPos_End = vCorner[i];
            _vector vFarPos_End = vCorner[i + 4];

            _vector vMyFarPos_Additive = {};
            _vector vMyNearPos_Additive = {};
            vMyFarPos_Additive = (vFarPos_End - vNearPos_End) * (m_fFar[j] - 0.1f) / 1000.f;
            vMyNearPos_Additive = (vFarPos_End - vNearPos_End) * (m_fNear[j] - 0.1f) / 1000.f;
            /*Near*/
            XMStoreFloat3(&m_vPoints_InWorld[j][i], vCorner[i] + vMyNearPos_Additive);
            /*Far*/
            XMStoreFloat3(&m_vPoints_InWorld[j][i+4], vCorner[i] + vMyFarPos_Additive);
        }
    }

    for (int i=0; i< CSM_END; ++i)
    Make_Planes(m_vPoints_InWorld[i], m_vPlanes_InWorld[i]);

    //place the corners in world -> slice frustum in camera spcae-> and * cmera world matrix-> we got frustum sliced in world
    return S_OK;
}

```

NDC 영역을 월드 프리스탐으로 변환

카메라 거리별 그림자 영역 분할 · 계속

```

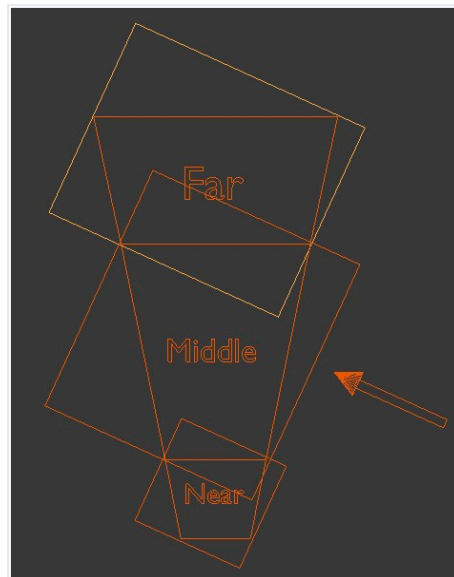
HRESULT CCSM_Manager::Compute_ViewMatrix()
{
    for (size_t i = 0; i < CSM_END; ++i)
    {
        _vector Center = XMVectorSet(
            m_BoundingAABBCorners[i].Center.x,
            m_BoundingAABBCorners[i].Center.y,
            m_BoundingAABBCorners[i].Center.z,
            1.f
        );
        // place the shadow camera toward reverse direction of The Light from Centre
        _vector vEyePos = Center - 2 * XMVector3Normalize(XMVectorSetf(XMLoadFloat3(&m_vLightDir), 0)) * m_BoundingAABBCorners[i].Extents.z;
        _vector vUp = XMVectorSet(0.f, 1.f, 0.f, 0.f);

        XMStoreFloat4x4(&m_TransformMatrices[D3DTS_VIEW][i], XMMatrixLookAtLH(vEyePos, Center, vUp));
        //EyePos->Center -> MakeViewMatrix
    }
    return S_OK;
}

HRESULT CCSM_Manager::Compute_ProjMatrix()
{
    for (size_t i = 0; i < CSM_END; ++i)
    {
        _float left = - m_BoundingAABBCorners[i].Extents.x;
        _float right = + m_BoundingAABBCorners[i].Extents.x;
        _float bottom = - m_BoundingAABBCorners[i].Extents.y;
        _float top = m_BoundingAABBCorners[i].Extents.y;
        _float nearZ = - m_BoundingAABBCorners[i].Extents.z;
        _float farZ = + m_BoundingAABBCorners[i].Extents.z;
        XMStoreFloat4x4(&m_TransformMatrices[D3DTS_PROJ][i], XMMatrixOrthographicOffCenterLH(left, right, bottom, top, 0.f, farZ - nearZ));
    }
    return S_OK;
}

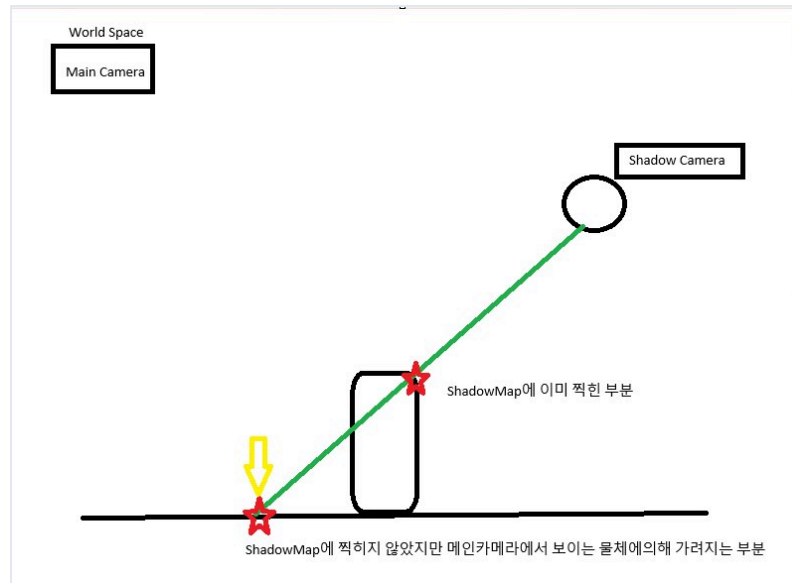
```

그림자 카메라의 뷰·투영 행렬 설정



픽셀 위치와 그림자 맵 깊이 비교 코드

카메라 거리별 그림자 영역 분할 · 계속



캐스케이드 그림자의 장면 출력

04 / 보스 전투

라이넬·가논의 내부 도메인 풀링과 공격 수명 관리

보스가 전용 공격 객체를 자식으로 보관하고, 종류별 순환 풀로 공격 실행과 후속 패턴을 연결했다.

[블로그 원문 ↗](#)

구현 구조

TEXT

```
Client/Private/Boss_Lynel.cpp
Client/Private/Ganon.cpp
Client/Private/Boss_Lynel_FSM.cpp
Client/Private/Ganon_FSM.cpp
```

보스 생성 시 공격 등록 → FSM 진입·유지·종료 → 종류별 풀 활성화 → 충돌·수명 종료 → 비활성화와 후속 공격.

상태별 진입·유지·종료

`CBossLynelFSM`과 `CGanonFSM`은 상태마다 진입·유지·종료 함수를 연결한다. 상태가 바뀌면 `ChangeState()` 경로에서 이전 상태를 빠져나가고 새 상태에 들어간다. 라이넬 객체는 몸체, 무기, 공격 범위와 FSM을 자식·컴포넌트로 연결한다. 몸체의 최종 월드 행렬과 무기 소켓 행렬을 전달해 공격 오브젝트가 보스의 자세를 따라가게 한다.

가논 FSM에는 대기, 전후좌우 이동, 방향 전환과 여러 공격 상태가 정의돼 있다. 천장·지상 위치 조건에 따라 사용할 수 있는 공격 경로도 달라진다. FSM은 상태에 따라 보스의 공격 활성화 함수를 호출한다.

라이넬·가논의 내부 도메인 풀링

라이넬과 가논은 전용 공격 객체를 보스 내부 도메인의 풀로 관리한다. 각 보스의 `m_Skills`가 공격 태그별 객체 배열을 보관하고, `m_SkillIndices`가 다음에 사용할 인덱스를 보관한다. 보스는 공격 객체를 자식으로 연결해 자신의 전투 흐름 안에서 재사용한다.

`CBossLynel::Ready_Skills()`는 화염, 검기, 영역 공격과 2페이지의 후속 폭발·화염 장판·화염 기둥을 준비한다.

`Register_Skill_Fire()`는 프로토타입을 복제하고 `Add_Child()`로 라이넬에 연결한 뒤 `m_Skills[TEXT("Pooling_S_Fire")]`에 보관한다. 등록 직후에는 공격 객체를 비활성화하고 해당 종류의 인덱스를 0으로 둔다.

화염 등록 시에는 라이넬 몸체의 최종 월드 행렬, 머리 소켓 행렬, `m_SkillSigns`의 주소를 전달한다. 공격 객체가 필요한 보스의 자세와 전투 신호를 보스 내부에서 연결하는 구조다. 공격 객체를 미리 구성해 두므로 공격 시점에는 프로토타입 복제와 자식 등록을 반복하지 않는다.

라이넬·가논의 내부 도메인 풀링과 공격 수명 관리 · 계속

Client/Private/Boss_Lynel.cpp 발췌

```

CPP
HRESULT CBossLynel::Register_Skill_Fire()
{
    CLynel_Fire::LYNEL_FIRE_DESC tDesc = {};
    tDesc.iCollisionGroup = ATK_COLL;
    tDesc.fSpeedPerSec = 15.f;
    tDesc.fRotationPerSec = XMConvertToRadians(90.f);
    tDesc._pBody_Float4x4 = static_cast<CBossLynelBody*>(Find_Child(TEXT("Body"))->Get_FinalWorldMatrix_Ptr());
    tDesc._pSocket_Float4x4 = m_vecChildren.front()->GetComponent<CNewModel>()->Get_BoneMatrix("Head");
    tDesc._pSkillSigns = &m_SkillSigns;
    _tchar szTag[MAX_PATH] = TEXT("Pooling_S_Fire%d");
    m_Skills[TEXT("Pooling_S_Fire")].resize(LYNEL_NUM_FIRES);
    for(size_t i=0;i< LYNEL_NUM_FIRES;++i)
    {
        _tchar szNowTag[MAX_PATH] = {};
        wsprintf(szNowTag ,szTag, i);
        Add_Child(static_cast<CGameObject*>(m_pGameInstance->Clone_Prototype(PROTOTYPE::PROTO_GAMEOBJ, LEVEL_GAMEPL
AY,
            TEXT("Prototype_GameObject_Lynel_Fire"), &tDesc)),szNowTag);
        m_Skills[TEXT("Pooling_S_Fire")][i] = m_vecChildren.back();
        ACTIVE_OBJ(m_Skills[TEXT("Pooling_S_Fire")][i], false);
    }
    m_SkillIndices.emplace(make_pair(TEXT("Pooling_S_Fire"), 0));
    return S_OK;
}

```

공격 종류별 순환 재사용과 실행 정보

라이넬의 정보 전달용 `Active_Skill()`은 다음 순서로 동작한다.

```

CPP
void CBossLynel::Active_Skill(_wstring SkillTag, int NumMaxSkills, void* pArg)
{
    _uint iIndex = m_SkillIndices.find(SkillTag)->second;
    static_cast<CBossSkill_Base*>(m_Skills[SkillTag][iIndex])->Set_Info(pArg);

    ACTIVE_OBJ(m_Skills[SkillTag][iIndex], true);
    m_SkillIndices[SkillTag] = (iIndex + 1) % NumMaxSkills;
}

```

보스는 공격 종류의 현재 슬롯에 실행 정보를 전달한 뒤 활성화한다. 다음 슬롯은 나머지 연산으로 순환한다. 위치와 크기처럼 공격마다 달라지는 값은 `Set_Info()`에 전달하는 descriptor로 갱신한다. 가논도 종류별 배열과 순환 인덱스로 전용 공격을 활성화한다.

풀을 통해 이어지는 보스 패턴

라이넬의 화염은 지면 충돌 시 `Sign_Lynel_2Phase`를 확인한다. 2페이지라면 화염 객체가 자신의 위치와 크기를 descriptor에 담고 부모 라이넬의 `Active_Skill()`을 호출한다. 라이넬은 내부 풀의 `Pooling_FireArea`와 `Pooling_FirePhillar`를 활성화한다. 화염 객체는 후속 공격을 연결한 뒤 자신을 비활성화한다.

라이넬·가논의 내부 도메인 풀링과 공격 수명 관리 · 계속

영역 공격인 `CLyne1_S_Field`도 수명이 끝날 때 2페이지 여부를 검사한다. 2페이지에서는 영역 공격이 부모 라이넬의 `Pooling_AfterBomb`을 활성화하고 종료한다. 보스의 풀은 공격 객체의 재사용과 함께 페이지에 따른 후속 공격 연결을 담당한다.

가논은 `AreSkillsAllInactive()`로 특정 종류의 공격이 모두 비활성화됐는지 검사한다. 얼음 공격 FSM은 누적 발사 수와 얼음 객체들의 비활성 상태를 조건에 포함한다. FSM의 진행 조건에 내부 풀의 실행 상태가 연결된 사례다.

피격이나 착지 효과에는 `ACTIVE_POOLING`을 통한 엔진 공통 풀 호출도 사용한다. 전용 공격은 보스 내부의 배열과 인덱스로 관리하고, 개별 효과의 활성화는 해당 효과가 등록된 풀을 사용한다.

패턴별 실행 장면과 조건은 [가논의 거리·페이지 전환](#)과 [라이넬의 순차 패턴·기절 처리](#)에서 이어진다.

가논의 공격 종료 확인

가논은 종류별 배열의 활성 상태를 순회한다. 해당 종류의 객체가 모두 비활성이면 FSM이 다음 단계로 넘어갈 때 사용하는 조건을 만족한다.

`Client/Private/Ganon.cpp` 발췌

```
CPP
bool CGanon::AreSkillsAllInactive(_wstring SkillTag)
{
    _bool Ret = { true };
    size_t SkillSize = m_Skills[SkillTag].size();
    for (size_t i = 0; i < SkillSize; ++i)
    {
        if (m_Skills[SkillTag][i]->Get_Active())
            Ret = false;
    }
    return Ret;
}
```

05 / 보스 전투

가논 — 거리 기반 공격과 벽·천장 전환의 행렬 회전

거리와 공격 횟수로 패턴을 선택하고, 체력에 따라 PhysX 중력과 물체 행렬을 제어해 지상·벽·천장 전투를 연결했다.

[블로그 원문 ↗](#)

구현 구조

TEXT

```
Client/Private/Ganon_FSM.cpp
Client/Private/Ganon.cpp
Client/Private/Ganon_Tornado.cpp
```

거리·공격 횟수·체력 검사 → 패턴 선택 → 지상·벽·천장 자세 전환 → 공격 풀 활성화.



가논 등장 컷신 · [MP4 직접 열기](#)

거리, 공격 횟수, 체력으로 전투 흐름 나누기

가논은 플레이어와의 거리로 공격을 고른다. 근접 공격을 일정 횟수 수행하면 플레이어에게서 멀어져 원거리 공격으로 넘어간다. 체력이 줄면 벽과 천장으로 이동하면서 전투 위치도 바뀐다. 공격 선택에는 거리와 누적 횟수가, 전투 공간 전환에는 체력과 자세 상태가 필요했다.

`CGanonFSM`에는 상태별 진입·유지·종료 함수를 연결했다. 전용 공격 객체는 가논 내부의 풀에 준비하고, FSM이 패턴에 맞춰 활성화한다. 객체 등록과 순환 재사용은 [보스 내부 도메인 풀링](#)에서 다룬다.

가는 — 거리 기반 공격과 벽 천장 전환의 행렬 회전 · 계속

지상: 거리와 좌우 방향으로 공격 선택하기

가까운 거리에서는 등 뒤의 칼로 순차 공격

플레이어가 가까우면 가는 등 뒤의 세 손에 달린 칼로 순서대로 공격한다. 가까운 거리의 연속 공격과 조금 떨어진 거리의 횡베기를 구분해, 같은 지상 상태에서도 거리에 따라 다른 공격을 선택하도록 했다.

거리가 벌어지면 횡베기와 세 갈래 공격

플레이어가 조금 더 멀어지면 횡베기를 수행하고 가는 앞에 세 개의 공격을 배치한다. `CGanon::Register_LSS_Pillar()`에서는 Y축 기준 20° , -20° , 0° 의 회전 행렬을 만들어 각 공격의 `_Rotation_Float4x4`에 전달한다. 하나의 방향을 그대로 복제하지 않고 각 공격에 방향 차이를 준 구성이다.

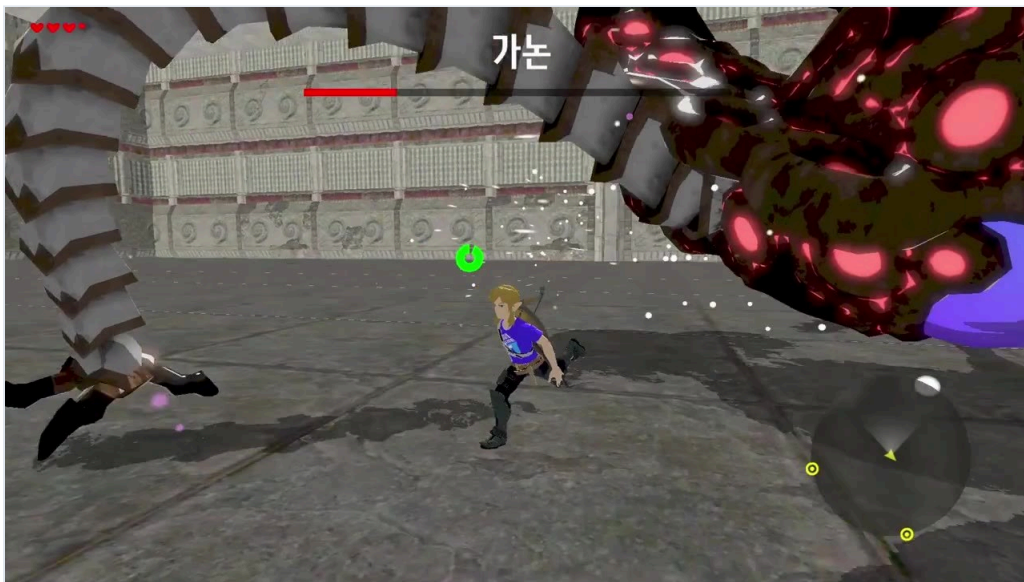
원거리에서는 외적으로 플레이어의 좌우 판별

플레이어가 가는 기준 오른쪽에 있으면 종베기를, 왼쪽에 있으면 레이저를 사용한다. 두 위치 사이의 거리만으로는 어느 쪽에 플레이어가 있는지 구분할 수 없어 방향 벡터의 외적을 이용했다.

근접 공격 후에는 가장 먼 지정 구역으로 이동

근접 공격 횟수가 기준을 넘으면 미리 지정한 구역 중 가장 먼 곳을 향해 이동하고 일정 거리에서 멈춘다.

`Ganon_MovB_PlacePosition()`은 지정 위치들과 가는 몸체 위치의 차이를 구한 뒤 Y 성분을 제외한다. 이 함수는 수평 거리가 가장 큰 위치를 `m_vDest`에 넣는다. 목적지 선택 기준은 가는 자신의 위치이며, 이후 이동으로 플레이어와의 거리를 벌려 원거리 패턴을 이어간다.



근접 공격을 마친 뒤 지정 구역으로 거리를 벌리는 가는 · MP4 직접 열기

체력 2/3 이하: 중력을 끄고 벽에 맞춰 몸체 회전

가는의 체력이 2/3 이하가 되면 벽으로 이동한다. 벽에 붙어 있는 동안에는 PhysX 중력을 끄고, 벽의 노멀 반대 방향이 몸체의 Up이 되도록 회전 보관한다. 이 단계에서는 위치 이동과 몸체의 방향 전환을 함께 다뤄야 한다.

`Ready_LerpUL()`은 기존 몸체 행렬의 Up-Look을 시작 벡터로 보관한다. 목표 Up에는 전달받은 레이 노멀의 반대 방향을, 목표 Look에는 $(0, 1, 0)$ 을 설정한다. `Lerp_Rotation_UL()`은 두 축을 보간·정규화한 뒤 외적으로 Right를 만든다.

가논 — 거리 기반 공격과 벽 천장 전환의 행렬 회전 · 계속

CPP

```
_vector UpLerp = XMVectorLerp(XMVector3Normalize(m_vSrcNormal),
    XMVector3Normalize(m_vDstNormal), t);
_vector LookLerp = XMVectorLerp(XMVector3Normalize(m_vSrcLook),
    XMVector3Normalize(m_vDstLook), t);
UpLerp = XMVector3Normalize(UpLerp);
LookLerp = XMVector3Normalize(LookLerp);
_vector RightNew = XMVector3Normalize(XMVector3Cross(UpLerp, LookLerp));
```

이 세 축을 몸체 행렬의 Right·Up·Look에 기록한다. 몸체 방향을 만드는 행렬의 이동 행은 $(0, 0, 0, 1)$ 로 두고, 가논 전체의 위치는 별도로 처리한다. FSM이 받는 `m_pBodyTransFloat4x4`는 가논의 Body 자식 트랜스폼을 가리킨다.



PhysX 중력을 끄고 벽 노면에 맞춰 회전하는 가논 · MP4 직접 열기

벽에서 스킬 패턴을 세 번 사용하면 지상으로 내려온다. 내려오는 전환에서는 $(0, 1, 0)$ 이 다시 Up이 되도록 자세를 보간해 지상 전투 자세로 돌아간다.



벽 패턴 종료 후 지상 자세로 돌아오는 가논 · MP4 직접 열기

가논 — 거리 기반 공격과 벽·천장 전환의 행렬 회전 · 계속

체력 1/3 이하: 벽에서 천장으로 전투 공간 확장

가논의 체력이 1/3 이하가 되면 벽에 붙은 상태에서 천장으로 이동한다. 천장 자세로 전환할 때도 몸체 행렬을 회전시키며, 전환을 마친 몸체의 Up은 다시 $(0, 1, 0)$ 방향을 향하도록 구성했다.

`Ready_LerpRU_Ceiling()`은 몸체의 Look을 축으로 -90도 회전 행렬을 만들고, 기존 Right·Up을 그 행렬로 변환해 목표 벡터를 구한다. `Ready_LerpRL_Ceiling()`은 몸체의 Up을 축으로 -90도 회전해 목표 Right·Look을 구한다. 전환에 필요한 축 조합에 따라 보간 함수를 나눴다.

Client/Private/Ganon_FSM.cpp 발췌

```

CPP

void CGanonFSM::Ready_LerpRU_Ceiling(_float fLerpSpeed)
{
    if (true == m_IsLerping) return;
    m_fAccLerpingTime = 0.f;
    m_fLerpSpeed = fLerpSpeed;
    _matrix MatRot = XMMatrixRotationAxis(XMLoadFloat3((_float3*)m_pBodyTransFloat4x4->m[2]), XMConvertToRadians(-90.f));
    m_vSrcRight = XMLoadFloat3((_float3*)m_pBodyTransFloat4x4->m[0]);
    m_vDstRight = XMVector3Normalize(XMVector3TransformNormal(m_vSrcRight, MatRot));
    m_vSrcNormal = XMLoadFloat3((_float3*)m_pBodyTransFloat4x4->m[1]);
    m_vDstNormal = XMVector3Normalize(XMVector3TransformNormal(m_vSrcNormal, MatRot));
    m_IsLerping = true;
}

```

함수	보간하는 두 축	외적으로 구하는 축
<code>Lerp_Rotation_UL()</code>	Up, Look	$\text{Right} = \text{Up} \times \text{Look}$
<code>Lerp_Rotation_RU()</code>	Right, Up	$\text{Look} = \text{Right} \times \text{Up}$
<code>Lerp_Rotation_RL()</code>	Right, Look	$\text{Up} = \text{Look} \times \text{Right}$

FSM은 전환 과정에서 `m_IsCeiling`과 `Ganon_OnCeiling`을 갱신한다. 천장에서 스킬을 세 번 사용한 뒤에는 PhysX 중력을 다시 적용해 떨어지도록 했다.

가논 — 거리 기반 공격과 벽·천장 전환의 행렬 회전 · 계속



벽에서 천장으로 이동하며 몸체 행렬을 회전하는 가논 · MP4 직접 열기



천장 공격 종료 후 중력을 다시 적용한 낙하 · MP4 직접 열기

얼음: 생성 예고와 메시 활성화 시점 연결

얼음 생성 영역은 가논 Up의 반대 방향에 배치한다. 가논이 지상·벽·천장 중 어디에 있는지에 따라 몸체의 축이 달라지므로, 생성 위치도 가논의 자세를 기준으로 정한다.

일정 시간이 지나면 얼음을 생성하고, 마법진이 페이드아웃되는 순간에 얼음 메시지를 렌더링하는 컴포넌트를 활성화한다. 이어서 얼음을 플레이어에게 던진다. 공격의 예고, 메시 표시, 발사를 각각의 시점에 연결한 흐름이다. 가논 FSM은 누적 발사 수와 `AreSkillsAllInActive()`의 결과를 함께 사용해 얼음 공격 진행 상태를 판단한다.

가논 — 거리 기반 공격과 벽·천장 전환의 행렬 회전 · 계속



마법진의 페이드아웃에 이어 열음이 나타나고 발사되는 공격 · MP4 직접 열기

구체: 소켓 위치와 목표 방향에 오프셋 적용

보라색 구체는 소켓 행렬로 생성 위치를 정한다. 가논에서 플레이어를 향하는 Look에 오프셋 벡터를 더해 여러 구체의 발사 방향을 만든다. 소켓은 공격이 출발하는 위치를, 목표 방향과 오프셋은 구체들이 퍼지는 방향을 결정한다.



소켓 위치에서 방향 오프셋을 주어 발사하는 구체 · MP4 직접 열기

토네이도: 전진과 좌우 이동을 함께 적용

토네이도는 가논의 Look 방향으로 전진하면서 좌우 이동을 함께 수행한다. 좌우 방향은 $\sin$ 값의 부호로 바꾼다.

[CGanon_Tornado](#)의 이동 계산에서는 초기 방향과 $\sin$ 값을 곱한 부호에 따라 방향을 선택하므로, 시작 방향을 달리한 토네이도도 같은 이동 계산을 사용할 수 있다.

$\sin$ 값은 좌우 전환 시점을 결정한다. 전진 성분에 부호로 선택한 횡이동 성분을 더해 공격이 좌우로 움직이며 다가오게 했다.

가논 — 거리 기반 공격과 벽 천장 전환의 행렬 회전 · 계속



전진하면서 sin 부호에 따라 좌우로 움직이는 토네이도 · MP4 직접 열기

06 / 보스 전투

라이넬 — 패턴 순환과 소켓 전환, PhysX 이동과 기절 처리

기본 공격과 순차 스킬을 연결하고, 소켓 행렬·애니메이션 이벤트·PhysX·충돌 판정으로 돌진과 점프, 기절, 2페이지 후속 공격을 구현했다.

[블로그 원문 ↗](#)

구현 구조

TEXT

```
Client/Private/Boss_Lyne1_FSM.cpp  
Client/Private/Boss_Lyne1.cpp  
Client/Private/Lyne1_Fire.cpp
```

패턴 순환 → 애니메이션 이벤트 → 소켓·물리 이동·공격 활성화 → 충돌과 착지 판정 → 2페이지 후속 공격.



라이넬 등장 컷신 · [MP4 직접 열기](#)

기본 공격 시간과 순차 스킬 패턴

라이넬은 스킬 패턴을 순서대로 사용하고, 일정량의 스킬을 사용한 뒤에는 기본 공격만 하는 시간을 갖는다. 같은 공격을 계속 반복하지 않도록 스킬 순서와 기본 공격 구간을 나눴다. 체력이 절반 이하가 되면 2페이지로 전환해 기존 공격에 후속 공격을 연결한다.

`CBossLyne1FSM`은 상태별 진입·유지·종료를 처리한다. 공격 객체는 라이넬 내부에 종류별로 준비하고 재사용한다. 이 글은 각 패턴이 어떤 위치·방향·충돌 조건을 사용하는지 다룬다. 풀의 등록과 활성화는 [보스 내부 도메인 풀링](#)으로 이어진다.

라이넬 — 패턴 순환과 소켓 전환, PhysX 이동과 기절 처리 · 계속

돌진: 칼을 휘두를 위치로 접근하기

칼을 들고 뛰어오는 공격에서는 플레이어 위치에서 라이넬 Right의 반대 방향으로 이동시킨 점을 접근 위치로 사용한다. 라이넬은 그 위치를 향해 달리고, 거리가 기준 이하가 되면 공격한다. 플레이어의 위치와 칼을 휘두르기 위한 접근 위치를 구분한 것이다.

라이넬의 Right는 몸체 자세에 따라 달라진다. 따라서 이 오프셋은 고정된 월드 X축이 아니라 공격하는 라이넬의 좌우 방향을 기준으로 계산한다.



플레이어 옆의 접근 위치로 달려와 칼을 휘두르는 라이넬 · MP4 직접 열기

점프: 물리 이동과 착지 공격의 시점 나누기

점프 공격에는 PhysX Impulse와 거리 기반 속도를 부여한다. 출발 시 힘을 주고 목표까지의 거리에 맞춰 이동 속도를 설정해 점프 이동을 만든다.

칼의 공격 활성화는 아래로 쏜 Ray가 지면과 충돌하는 시점에 연결했다. 점프를 시작한 시점과 실제로 내려찍는 시점은 다르므로, 지면 검출 결과로 착지 공격을 활성화한다.

라이넬 — 패턴 순환과 소켓 전환, PhysX 이동과 기절 처리 · 계속



PhysX로 점프한 뒤 지면 Ray 판정으로 내려찍기 공격을 활성화하는 라이넬 · MP4 직접 열기

Client/Private/Boss_Lynel_FSM.cpp 발췌

```

CPP
void CBossLynelFSM::OnJump_Stay()
{
    if (true == m_bJumpState[LYNEL_JUMP_IMPULSE])
    {
        m_bJumpState[LYNEL_JUMP_IMPULSE] = false;
        _float Length = {};
        _float4x4 PlayyerFloat4x4 = (m_pTarget)->Get_FinalWorldMatrix();
        _vector vTargetPos = { PlayyerFloat4x4._41,0.f,PlayyerFloat4x4._43,1.f };
        _float4x4 vMyFloat4x4 = m_pOwner->Get_FinalWorldMatrix();
        _vector vMyPos = { vMyFloat4x4._41,0.f,vMyFloat4x4._43,1.f };
        Length = XMVectorGetX(XMVector3Length(vTargetPos - vMyPos));
        m_pMovement->Move_Forward(Length*0.7f);
        m_pRigid_Owner->Add_Force(_float3(0.f,2000.f, 0.f), PxForceMode::eIMPULSE);
        m_bJumpState[LYNEL_JUMP_START] = false;
    }

    _float4x4 Myfloat4x4 = m_pOwner->Get_FinalWorldMatrix();
    _float3 MyPos = { Myfloat4x4._41 ,Myfloat4x4._42  -.5f, Myfloat4x4._43 };
    RaycastHit Out = {};
    m_pGameInstance->Raycast(MyPos, _float3(0.f, -1.f, 0.f), Out, 1.5f , 1<<FLD_COLL |1 << GROUND_COLL);
    if (nullptr != Out.pObj)
    {
        m_pAnimController_Owner->Set_Trigger(TEXT("tAttack"));
    }
}

if (true == m_bJumpState[LYNEL_JUMP_START] ||true == m_bJumpState[LYNEL_JUMP_LAND])

```

라이넬 — 패턴 순환과 소켓 전환, PhysX 이동과 기절 처리 · 계속

CPP · 계속

```
{
    m_pMovement->Stop_XZ();
}
if (m_bJumpState[LYNEL_JUMP_CMPL])
{
    m_Lynel_PreCond = m_Lynel_Cond;
    m_Lynel_Cond.iAct = LYNEL_AC_WAIT;
    ChangeState(m_Lynel_Cond.iAct);
}
}
```

기어 달리기: 이벤트로 무기 소켓 바꾸기

라이넬이 기어서 달리는 패턴에서는 칼의 장착 위치가 바뀐다. 애니메이션 이벤트 콜이 발생하면 칼이 참조하는 소켓 행렬을 변경해 장착 부위를 전환한다. 같은 칼 객체가 패턴 중 다른 부위를 따라가도록 만든 구성이다.

추적 방향은 플레이어 위치를 외적으로 판별해 정한다. 이동 중 목표가 좌우로 바뀌면 라이넬도 그 방향을 따라 회전하며 접근한다.



무기 장착 소켓을 바꾸고 플레이어를 추적하는 돌진 · [MP4 직접 열기](#)

화염 투사체: 머리 위치에서 준비하고 이벤트로 발사

화염 투사체의 출발점은 라이넬 머리 위치다. 투사체의 Look은 그 위치에서 플레이어를 향하도록 설정한다. 이후 애니메이션 이벤트 콜이 발생하면 투사체가 이동한다.

투사체를 준비하는 순간과 발사하는 순간을 나눠, 머리 앞에서 공격을 준비하는 동작과 실제 발사 시점을 연결했다. 공격 객체 등록 시에는 몸체의 최종 월드 행렬과 Head 소켓 행렬을 전달한다.

라이넬 — 패턴 순환과 소켓 전환, PhysX 이동과 기절 처리 · 계속



라이넬 머리에서 플레이어를 조준한 뒤 이벤트에 맞춰 발사하는 화염 · MP4 직접 열기

회전: Look과 목표 방향의 외적으로 좌우 판별

라이넬에서 플레이어를 향하는 방향과 라이넬의 Look을 외적한다. 이 구현의 판정 기준에서는 값이 양수면 시계 방향, 음수면 반시계 방향으로 회전한다. 플레이어가 어느 쪽에 있는지에 따라 회전 방향을 선택하는 계산이다.

외적은 두 벡터의 순서를 바꾸면 부호가 반대로 바뀐다. 따라서 이 부호 규칙은 벡터 순서와 좌표계가 함께 정해진 판정으로 사용한다.



플레이어 방향과 Look의 외적으로 회전 방향을 정하는 라이넬 · MP4 직접 열기

거리를 벌리는 두 가지 이동

백스텝은 PhysX Impulse와 Look 반대 방향의 속도를 부여해 순간적으로 뒤로 이동시킨다. 몸체가 플레이어를 바라보는 상태에서 빠르게 거리를 벌리는 동작이다.

뒤로 돌아 뛰어가는 패턴에서는 라이넬 Look을 플레이어를 바라보는 방향의 반대로 서서히 회전시킨다. 라이넬은 회전하면서 앞으로 이동해 플레이어에게서 멀어진다. 순간적인 후방 이동과 몸을 돌려 달리는 이동에 서로 다른 처리 방식을 적용했다.

라이넬 — 패턴 순환과 소켓 전환, PhysX 이동과 기절 처리 · 계속

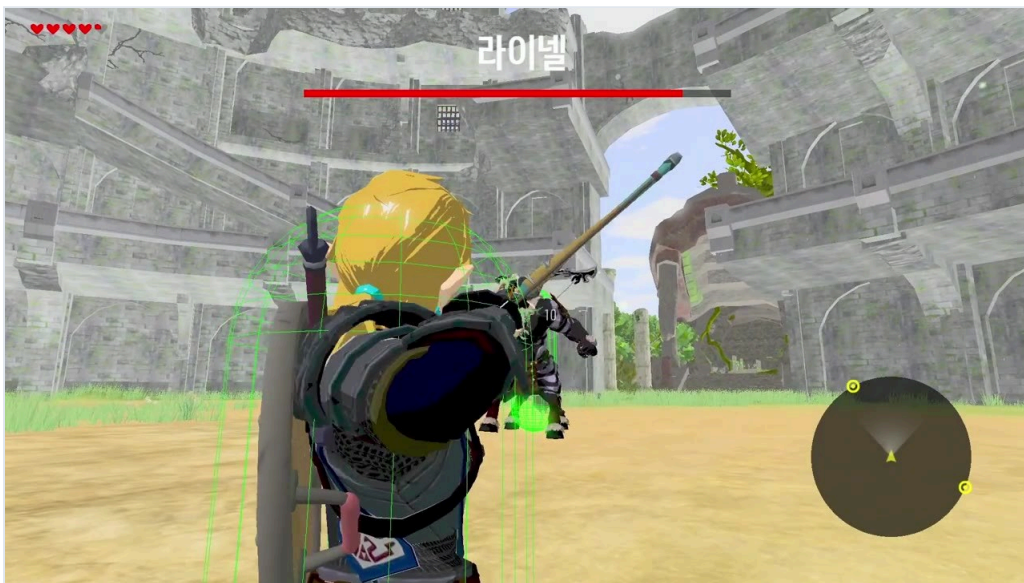
체력 절반 이하: 이벤트로 2페이지 연출 연결

라이넬의 HP가 절반 이하가 되면 2페이지를 시작한다. 전환 동작의 이벤트 콜에 맞춰 사운드와 이펙트 함수를 호출한다. 체력 조건은 페이지 전환을 결정하고, 애니메이션 이벤트는 전환 동작 안에서 연출이 실행될 시점을 결정한다.

머리 피격 기절과 중복 기절 방지

라이넬 머리와 플레이어 화살이 충돌하면 라이넬의 행동을 멈춘다. 기절 상태에서 TimeDelta를 누적하고 기준 시간을 넘으면 전투를 재개한다. 진입은 충돌 사건으로, 종료는 누적 시간으로 판단한다.

기절 중에는 다시 기절하지 않도록 기절 판정에 쓰는 콜라이더를 비활성화한다. 머리의 재피격을 계속 기절 진입으로 처리하면 기절 상태가 반복될 수 있으므로, 기절 상태 자체가 다음 기절 판정을 제한하게 했다.



머리 화살 피격에 따른 기절과 콜라이더 비활성화 · MP4 직접 열기

Client/Private/Boss_Lynel_FSM.cpp 발췌

```
CPP
void CBossLynelFSM::OnStun_Begin()
{
    if (true == m_IsSwordDrawn)
        Event_DrawSword();
    ACTIVE_OBJ(m_pOwner->Find_Child(TEXT("Body"))->Find_Child(TEXT("Head")), false);
    m_pAnimController_Owner->Set_Integer(TEXT("iAct"), LYNEL_AC_STUN);
    m_pAnimController_Owner->Set_Trigger(TEXT("tNxtAnim"));
    m_pGameInstance->Start_SFX(TEXT("Lynel_Vo_Down_01"), false);
}
```

2페이지 점프: 칼의 월드 행렬로 후속 구체 배치

2페이지에서는 점프 내려찍기를 마친 뒤 불꽃 구체가 생긴다. 구체에 칼의 월드 행렬을 전달하고, 구체 자체의 트랜스폼을 조절해 최종 위치를 정한다. 공격 기준 위치는 칼이 제공하고 구체의 개별 배치는 구체 트랜스폼이 담당한다.

점프의 착지 판정, 칼의 공격 활성화, 후속 구체 배치가 순서대로 이어진다.

라이넬 — 패턴 순환과 소켓 전환, PhysX 이동과 기절 처리 · 계속



점프 내려찍기 후 칼의 위치를 기준으로 생성되는 불꽃 구체 · MP4 직접 열기

2페이지 화염: 충돌 위치에 불기둥 활성화

화염 투사체가 땅에 닿으면 그 투사체의 월드 위치를 기준으로 불기둥을 활성화한다. `CLynel_Fire::CheckRays()`는 지면 충돌과 `Sign_Lynel_2Phase`를 확인한다. 이어서 투사체 트랜스폼의 위치를 `InitialPos`에 담고 부모 라이넬의 `Active_Skill()`에 전달한다.

라이넬은 내부 풀에서 `Pooling_FireArea`와 `Pooling_FirePhillar`를 활성화한다. 투사체는 월드 위치와 필요한 크기를 `descriptor`로 전달하고, 후속 공격은 이 정보로 배치된다. 화염 투사체는 후속 공격을 연결한 뒤 비활성화된다.



화염 투사체의 지면 충돌 위치에서 이어지는 불기둥 · MP4 직접 열기

2페이지 기본 공격: Look을 회전축으로 검기 방향 만들기

검기 객체에는 공격에 맞는 회전 각도를 전달한다. 검기는 라이넬 Look을 축으로 그 각도만큼 회전한 뒤 이동한다. 보스의 전방 방향을 유지하면서 베기 각도에 맞춰 검기의 자세를 바꾸는 처리다.

라이넬 — 패턴 순환과 소켓 전환, PhysX 이동과 기절 처리 · 계속



라이넬 Look을 축으로 회전한 뒤 이동하는 검기 · [MP4 직접 열기](#)